

Practical Session 3: LoRA, Scaling, and Fine-Tuning

Hands-On: Parameter Counting, Chinchilla Budgeting, and FinBERT Sentiment

Juan F. Imbet

Paris Dauphine – PSL University

Large Language Models in Finance

Session Overview

Duration: 1 hour

- ➊ **Recap** (5 min) — LoRA formula, Chinchilla rule, CLM/MLM objectives
- ➋ **Problem 1** (15 min) — LoRA parameter counting on 7B LLaMA
- ➌ **Problem 2** (10 min) — Chinchilla compute budgeting
- ➍ **Case Study** (20 min) — Fine-tune FinBERT sentiment with PEFT/LoRA; answer three diagnostic questions
- ➎ **Discussion** (5 min) — When to choose each adaptation strategy?
- ➏ **Solutions** (5 min) — Walkthrough of Problems 1 and 2

What you will practise

Translating scaling-law arithmetic into practical hardware decisions; applying LoRA configuration choices; running a real fine-tuning loop on a finance task.

Recap: LoRA Formula

For a frozen pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$:

$$W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

Trainable parameter count per adapted layer:

$$\Delta\theta = r(d + k)$$

Key properties:

- W_0 is frozen; only B and A receive gradients
- Initialise: $A \sim \mathcal{N}(0, \sigma^2)$, $B = 0$ so $BA = 0$ at step 0
- At inference: merge $W \leftarrow W_0 + BA$ — zero overhead vs. base model
- Rank r is the key hyperparameter; typical values: 4, 8, 16, 32

You will need this for Problem 1

Summing $r(d + k)$ over all adapted layers gives total trainable parameter count.

Recap: Chinchilla Rule and Pre-training Objectives

Chinchilla compute-optimal allocation:

$$C \approx 6ND, \quad \frac{D^*}{N^*} \approx 20$$

Given budget C : solve $6N(20N) = 120N^2 = C$, so

$$N^* = \sqrt{\frac{C}{120}}, \quad D^* = 20N^*$$

Pre-training objectives (summary):

Objective	Architecture	Finance use
CLM (causal)	Decoder-only	Generation, chat
MLM (masked)	Encoder-only	Classification, NER
Span corruption	Encoder-decoder	QA, summarisation

You will need these for Problem 2

The formula $N^* = \sqrt{C/120}$ is your main tool for Chinchilla budgeting.

Problem 1: LoRA Parameter Counting — Setup

Model: LLaMA-7B with 32 transformer layers, hidden dimension $d = 4096$.

Each layer contains four linear projection matrices:

- Query projection: $W_Q \in \mathbb{R}^{4096 \times 4096}$
- Key projection: $W_K \in \mathbb{R}^{4096 \times 4096}$
- Value projection: $W_V \in \mathbb{R}^{4096 \times 4096}$
- Output projection: $W_O \in \mathbb{R}^{4096 \times 4096}$

Task: Apply LoRA with rank $r = 4$ to the Q and V projections only.

Fill in the table below for one layer:

Layer	Matrix	d	k	r
Transformer ℓ	W_Q			
Transformer ℓ	W_V			

Hint: For each adapted matrix the trainable param count is $r(d + k)$. Sum over both matrices and all 32 layers.

Problem 1: Tasks

Using the setup from the previous frame ($r = 4$, Q and V only, 32 layers, $d = k = 4096$):

Task A. Compute the total number of trainable LoRA parameters. Show the step-by-step arithmetic.

Task B. Express this as a fraction of the 7B total parameter count. Is the fraction above or below 0.1%?

Task C. Suppose you double the rank from $r = 4$ to $r = 8$. By what factor does the trainable parameter count change? Is the relationship linear, quadratic, or something else? Justify.

Task D. A colleague suggests using adapter modules instead of LoRA because adapters have been around longer. Give one concrete reason why LoRA is preferred for *inference-time deployment* in a high-throughput trading-floor application where latency is measured in milliseconds.

Time: 15 minutes. Work individually or in pairs.

Problem 2: Chinchilla Compute Budgeting

Scenario: An investment bank's AI team has a compute budget of $C = 10^{22}$ FLOPs for training a financial language model.

Using $C \approx 6ND$, $D^* \approx 20N^*$, and therefore $N^* = \sqrt{C/120}$:

Task A.

- 1 Compute N^* and D^* .
- 2 The team has assembled a financial text corpus containing 50B tokens. Is the team data-constrained or compute-constrained? (Hint: compare your D^* to 50B tokens.)
- 3 What should they do given this diagnosis?

Problem 2: Chinchilla Compute Budgeting (continued)

Task B. The team instead decides to train a 1B-parameter model. Using the Chinchilla rule, how many tokens should they train on for a compute-optimal run? What is the implied FLOP cost?

Task C. A senior researcher argues “we should train the biggest model possible to maximise accuracy”. Under what conditions is this argument valid? Under what conditions is it wrong?

Time: 10 minutes.

Case Study: Fine-Tuning FinBERT with LoRA on Financial Sentiment

Goal: Load a pre-trained FinBERT model and apply LoRA fine-tuning on a small set of labelled financial sentences using HuggingFace PEFT.

Stack:

- `transformers` — load `ProsusAI/finbert`
- `peft` — apply `LoraConfig`, wrap model with `get_peft_model`
- `torch` — training loop (no Trainer class for transparency)

Case Study: FinBERT with LoRA — What You Will Do

Dataset: 5 labelled financial sentences (positive/negative/neutral). This is intentionally tiny — the goal is to observe the mechanics, not maximise accuracy.

What you will do

- 1 Run the code on the next frame
- 2 Observe how many parameters are trainable vs. frozen
- 3 Answer the diagnostic questions on the following frame

Prerequisites

```
pip install transformers peft torch
```

No API key required for this case study.

Case Study: LoRA on FinBERT — Code (Part 1: Setup)

```
import torch
from transformers import AutoTokenizer,
    AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model, TaskType

MODEL = "ProsusAI/finbert"
SENTENCES = [
    ("Revenue increased 15% year-over-year.", 0),          #
    positive
    ("The firm reported a net loss of $200M.", 1),         #
    negative
    ("Operating margins remained broadly stable.", 2),     #
    neutral
    ("Strong demand drove record quarterly earnings.", 0), #
    positive
    ("Guidance was withdrawn amid macro uncertainty.", 1), #
    negative
]

tokenizer = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForSequenceClassification.from_pretrained(MODEL)
```

Case Study: LoRA on FinBERT — Code (Part 2: Train + Eval)

```
texts, labels = zip(*SENTENCES)
enc = tokenizer(list(texts), padding=True, truncation=True,
                return_tensors="pt")
label_tensor = torch.tensor(labels)

optimizer = torch.optim.AdamW(model.parameters(), lr=2e-4)
model.train()
for epoch in range(3):
    out = model(**enc, labels=label_tensor)
    out.loss.backward(); optimizer.step(); optimizer.
        zero_grad()
    print(f"Epoch {epoch+1}: loss={out.loss.item():.4f}")

model.eval()
with torch.no_grad():
    logits = model(**enc).logits
    preds = logits.argmax(-1)
    id2label = {0: "positive", 1: "negative", 2: "neutral"}
    for sent, pred in zip(texts, preds):
```

Case Study: Diagnostic Questions

Run the code and observe the output, then answer:

Q1. `model.print_trainable_parameters()` reports the number of trainable parameters. Compare this to the total FinBERT parameter count ($\approx 110\text{M}$ for BERT-large variants). How does the trainable fraction compare to the theoretical value for LoRA $r = 8$ applied to query and value matrices across all 12 BERT layers?

Q2. Change $r=8$ to $r=4$ and then to $r=16$. Run the three-epoch training loop for each. Do the final predictions change? Does loss decrease faster at higher rank? What does this suggest about the relationship between rank and expressiveness for this small dataset?

Q3. Modify `target_modules` to include all four projections: `["query", "key", "value", "dense"]`. How does the trainable parameter count change? Do the predictions improve or change? Is the additional compute justified?

Optional extension

Discussion: Choosing the Right Adaptation Strategy

For each scenario below, identify which strategy you would use (zero-shot / few-shot ICL / LoRA fine-tune / full fine-tune) and why.

- 1 A hedge fund needs to classify 10-K risk-factor sections as “material” or “immaterial” for a specific covenant type. They have 50 labelled examples and a 2-hour deadline. They use GPT-4o via API.
- 2 A tier-1 bank wants to build an internal model for earnings-call sentiment that runs on-premise (no external API), processes 200,000 documents per month, and must meet GDPR data-residency requirements. They have 20,000 labelled examples.
- 3 A fintech startup is building a general-purpose financial chatbot. They have no task-specific labelled data but have access to a pre-trained instruction-tuned LLM.
- 4 A quantitative research team wants to test whether a 70B model can replicate the reasoning of a domain expert on novel bond structuring questions. No labelled data exists.

Frame your answer around:

Solution: Problem 1 — LoRA Parameter Counting

Filled table (one layer, $r = 4$, $\mathbf{Q}$ and $\mathbf{V}$, $d = k = 4096$):

Layer	Matrix	d	k	r
Transformer ℓ	W_Q	4096	4096	4
Transformer ℓ	W_V	4096	4096	4

Task A:

$$32 \times 2 \times r \times (d + k) = 32 \times 2 \times 4 \times 8192 = 2,097,152 \approx 2.1\text{M params}$$

Task B: $2.1\text{M}/7000\text{M} \approx 0.030\%$ — well below 0.1%.

Task C: Doubling r from 4 to 8:

$$32 \times 2 \times 8 \times 8192 = 4,194,304 \approx 4.2\text{M}$$

The count *doubles* (linear in r , since $r(d + k)$ is linear in r).

Task D: LoRA merges $W_0 + BA$ into a single weight before deployment, so the merged model is identical in structure to the base model. Adapters add a sequential bottleneck module that cannot be trivially removed, increasing inference latency per token — unacceptable in latency-sensitive

Solution: Problem 2 — Chinchilla Budgeting + Wrap-Up

Task A ($C = 10^{22}$):

$$N^* = \sqrt{\frac{C}{120}} = \sqrt{\frac{10^{22}}{120}} = \sqrt{8.33 \times 10^{19}} \approx 2.89 \times 10^8 \approx 288\text{M params}$$

$$D^* = 20 \times N^* \approx 20 \times 288\text{M} = 5.76\text{B tokens}$$

The team has 50B tokens but $D^* \approx 5.76\text{B}$. $50\text{B} \gg 5.76\text{B}$: they are *compute-constrained*, not *data-constrained*. They should train a $\approx 288\text{M}$ -parameter model on 5.76B tokens; the remaining 44B tokens support a larger model if compute permits.

Task B (1B-parameter model): $D^* = 20 \times 1\text{B} = 20\text{B tokens}$;
 $C = 6 \times 1\text{B} \times 20\text{B} = 1.2 \times 10^{23}$ FLOPs.

Task C: Maximise-parameter argument is valid only when inference cost is negligible and compute is unlimited. In practice, the 50B-parameter model costs more to serve at inference time than a Chinchilla-optimal 288M model, and may *score worse* on held-out loss if under-trained.

APPENDIX

Stretch Problem: DPO on a Preference Dataset

Extra / optional material — beyond the core lecture.

Stretch Problem: From LoRA Fine-Tuning to DPO

Setup. You have a LoRA-fine-tuned FinBERT-style sentiment model from the case study. Now consider aligning a small *generative* finance model with Direct Preference Optimisation (DPO) on preference pairs (x, y_w, y_l) .

Task A. Recall the DPO objective:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Explain, in one sentence, what raising β does to how far π_{θ} may drift from π_{ref} .

Task B. Why does DPO need a frozen reference policy π_{ref} at all? What failure occurs if you drop it?

Task C. Construct three plausible $(y_w \succ y_l)$ preference pairs for a financial-advice prompt where “plausible vs. accurate” is the distinguishing axis.

Time: 15 minutes (optional / take-home).